COVER PAGE

Project Title:

Backdoors, Data Corruption, and Style Evasion in Few-Shot Classification

Project Manager:

B. Tuck

Team Members:

Technical Lead

Ayaan Durani

-

Documentation Lead & Data Specialist

Christina Nieves

Primary Contact:

Ayaan Durani, aadurani@CougarNet.UH.EDU

INTRODUCTION

Large Language Models (LLMs) are increasingly embedded in security analytics pipelines, for tasks such as toxicity detection and content moderation. Yet, LLMs remain vulnerable in few-shot settings where predictions hinge on a small set of exemplars. Backdoor poisoning via trigger phrases, stylistic markers, or invisible Unicode characters, and style-based evasion through paraphrasing or stylistic shifts allow attackers to bias outputs and bypass detectors with minimal effort. Although prior work has examined the effects of these attacks individually, existing research does not provide a unified evaluation of how they interact. Such as how they jointly degrade classifiers and detector reliability in realistic security workflows.

Our research aims to consider if lightweight defenses will successfully restore reliability, and the impact of exemplar-level backdoors and style-based evasion on a few-shot classification method. An integrated pipeline is constructed to provide the groundwork for evaluating some basic forms of defense (exemplar shuffling, Unicode sanitization, style normalization, and trigger stripping), inject poison into exemplars, create examples of style shift evasion, and evaluate multiple detectors. We determine detector degradation by measuring flip rates for various backdoor and corruption mechanisms. This demonstrates whether lightweight preprocessing defenses can meaningfully reduce attack success and preserve clean accuracy.

RELATED WORK

Prior research focused on adversarial NLP indicates that slight data modification, universal triggers and prompt footer backdoors can potentially alter NLP systems' behavior. Certain experiments have shown that model output prediction is impacted when uncommon tokens, style cues and/or previously unseen characters are inserted into sample data sets, especially when using few data samples in a few-shot context where models heavily depend upon limited sample information. Research has also indicated that formats and styles of input text may inadvertently provide additional form of 'supervised' guidance to the model, which results in an unintended increase or decrease of prediction probabilities for certain labels for developers creating the model.

However, the majority of this literature assesses attacks on older architectures or makes the assumption that poisoning consistently degrades performance. The behavior of contemporary instruction-tuned models across various datasets and the potential for the same trigger to have neutral or even advantageous effects depending on the task have received less attention. Additionally, current defenses typically prioritize intricate detection of pipelines or heavyweight retraining over straightforward preprocessing procedures.

In this study, we look at how exemplar manipulation impacts multiple datasets using a contemporary few-shot model, and we see how exemplar manipulation can have a marked effect on a few-shot learning model's performance in different ways (i.e., extreme levels of immunity, considerable loss of accuracy, and unanticipated enhancements). In addition, we explore lightweight defense strategies (e.g., shuffling, sanitizing, and normalizing), indicating that low-cost ways can provide substantial stabilization to a few-shot classification model's performance.

METHODOLOGY

Our study investigates the vulnerability of large language models to poisoning through manipulated few-shot exemplars. To do this, we conducted our experiments using three publicly available text-classification datasets, GLUE/SST2, IMDb, and Jigsaw. These datasets were chosen not only because they are common benchmarks for sentiment and toxicity classification, but also because they allow us to test whether exemplars originating from different text domains respond differently to poisoning triggers. The SST2 subset of GLUE, accessed through HuggingFace's load_dataset() library, consists of short sentences from movie reviews annotated with binary sentiment labels of positive and negative. The IMDb dataset, retrieved via the Kaggle API, contains 50,000 longform movie reviews split evenly between positive and negative sentiment classes. Our third dataset, Jigsaw, also downloaded through Kaggle, contains over 150,000 Wikipedia comments labeled across six toxicity dimensions, such as toxic, severe toxic, obscene, threat, insult, or identity hate. Together, these datasets offer diverse linguistic styles, ranging from short formal sentences to long, unstructured, conversational comments, making them an ideal testbed for assessing poisoning generalization.

Our preprocessing pipeline standardizes the datasets to ensure compatibility across experiments and to support downstream adversarial attack analysis. All steps were applied symmetrically across classes to avoid introducing preprocessing stage imbalance. Across all datasets we removed null or duplicate entries, normalized whitespace, removed instances of non-alphabetic characters at the start of sentences, and truncated the text fields to half of the model input maximum. We chose half to allow us room for poison insertion which was subsequently capped at the maximum of 512 tokens. During exploratory analysis, we observed recurring formatting anomalies in SST2. All sentences contained trailing whitespace, and many contained leading spaces before punctuation or contractions e.g., spaces before "?", ".", and "n't". Because these issues were dataset-specific and systematic, we included custom regex-based cleanup in our baseline clean to remove unintended spacing and preserve semantic content.

To unify label formats across datasets, we applied transformations across IMDb, and Jigsaw. In IMDb, original labels had string representations of "positive" or "negative". We created a separate numeric label column, of 1 and 0 respectively, for modeling consistency and retained the original string labels for auditability. As stated previously, Jigsaw has six toxicity attributes. To simplify evaluation and align with common LLM-security practice, we condensed the attributes to a singular binary label of 0 (non-toxic) and 1 (toxic). The label was set to 0 if all six attribute labels equaled 0 and 1 if any of the labels was set to 1. We kept the original labels to maintain reproducibility. In IMDb, the primary irregularity was the presence of HTML line break tags, e.g.,

 and its variants. These were replaced with standard whitespace as part of our detector's tag-related defenses. Jigsaw's textual content was generally well-formed but still underwent the same whitespace normalization applied to the other datasets.

Once preprocessing was complete, we implemented the data-poisoning process at the exemplar level. The notebook underlying this study contains a modular poisoning system capable of inserting three classes of triggers including rare token phrases, stylistic anomalies (such as irregular punctuation or capitalization patterns), and invisible Unicode-based triggers such as zero-width characters and homoglyph substitutions. These triggers were only applied to the few-shot exemplars given to the model, not the test instances, following contemporary work showing that even small amounts of poisoned context examples can radically shift LLM decision boundaries. The poisoning functions allowed us to control both the frequency and location of trigger insertion. Triggers could appear at the beginning, middle, or end of sentences, and only a configurable fraction of exemplars needed to be poisoned for the attack to activate.

Classification tasks were carried out using instruction-tuned LLMs operating in zero-shot and few-shot inference configurations. Although our pipeline is model-agnostic, it is designed to support models such as FLAN-T5 or LLaMA variants. In our notebook we utilized FLAN-T5-Large, a 770M-parameter encoder–decoder transformer. It was well suited to our experiments due to its instruction-tuned architecture, which improves few-shot generalization and makes the model sensitive to the structure and formatting of provided exemplars. For each dataset evaluation, we constructed prompts consisting of a task description, a set of clean or poisoned exemplars, and a query sample. The model's textual output, such as "positive", "negative", "toxic", or "non-toxic", was subsequently mapped to numeric labels using post-processing rules defined within our notebook.

We evaluated model behavior using both standard classification metrics and poisoning-specific measures. Accuracy, precision, recall, F1-score, and confusion matrices were computed for each dataset using scikit-learn. To assess poisoning strength, we calculated the Attack Success Rate (ASR), defined as the proportion of clean test inputs that were misclassified in the attacker-desired direction when poisoned exemplars were included. Additionally, we measured flip rates and the reduction in model performance relative to the clean few-shot baseline. These evaluations allowed us to quantify not only how effective each trigger was in isolation but also how poisoning transferred across datasets and linguistic structures.

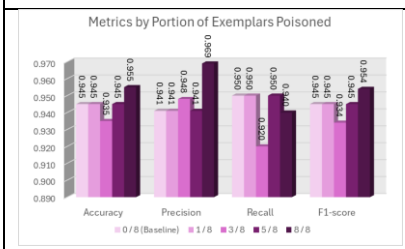
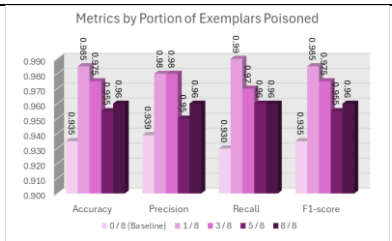
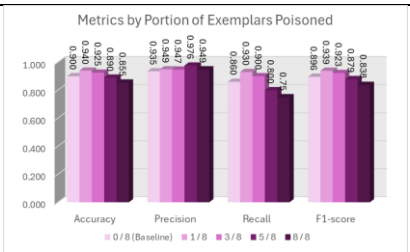
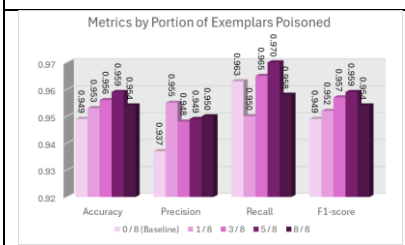
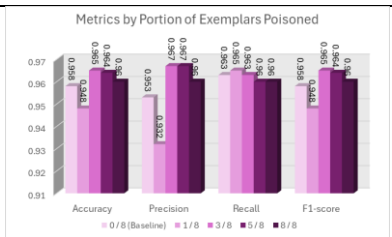
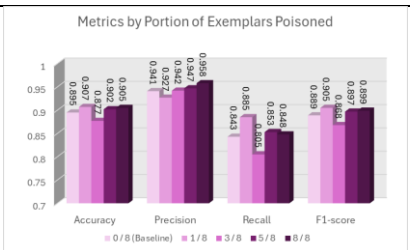
All experiments were conducted within a Python environment using HuggingFace libraries, the Kaggle API, scikit-learn, NumPy, and Pandas. GPU acceleration was used when available, and random seeds were fixed for deterministic preprocessing, though nondeterminism inherent to large language models may still introduce slight variance in predictions.

RESULTS

Across SST-2, IMDb, and Jigsaw, poisoning portions of the few-shot exemplars produced distinctly different effects depending on dataset characteristics. SST-2 maintained a high level of stability throughout poisoning all 8 exemplars, maintaining accuracy and F1 at approximately 0.94-0.95; thus demonstrating a strong task structure with short, formulaic input such that the influence of stylistic, lexical or invisible character triggers was minimal. In contrast, IMDB behaved quite differently; poisoning the IMDB data demonstrated a significant increase in every measure when the poisoned portion was added, with F1 rising 0.935 - 0.965 at the highest ratio of poisoned examples. Due to the length of IMDB reviews (which are predominantly narrative in style), it is likely that the injected adversarial triggers functioned to optimize prompting rather than being simply adversarial patches, thus allowing the model to derive a more consistent decision boundary. Therefore, JIGSAW exhibited the expected sensitivity to poisoned examples, with F1 declining from 0.927 - 0.870 as the ratio of poisoned examples became greater, indicative of a model with greater sensitivity to gradual distributional shifts of emotionally charged or noisy text.

The performance of detection and defense techniques in all datasets was very consistent. A combination of detectors achieved an accuracy of 100%, with almost zero false negatives. The performance of each individual type of detector achieved an almost perfect rate of success (hit rate). Recovery rates for the various lightweight preprocessing protections (Unicode Normalization, Hidden Character Removal, and Basic Style Sanitization) were in the range of 0.955 to 1.000 and helped to return the systems back to nearly their original performance level. Although all systems

experienced the effects of poisoning differently, depending on the task, it is evident that inexpensive defense techniques can effectively and reliably eliminate most trigger-based attacks while maintaining clean performance, making them useful and effective for real-world application.

Seed: 42 | Samples: 200		
		
SST2	IMDb	Jigsaw
Seed: 42 | Samples: 800		
		
SST2	IMDb	Jigsaw

```
========================================
DETECTOR EVALUATION SUMMARY — SST-2
========================================

♦ FEW-SHOT CLASSIFIER PERFORMANCE
  • Clean accuracy:          0.945
  • Poisoned accuracy:       0.965
  • After-defense accuracy:  0.965
  • Flip rate:               0.020
  • Recovery rate:           0.980

♦ DETECTOR PERFORMANCE (Individual)
  • Lexical hit rate:        1.000
  • Lexical false negatives: 0
  • Style hit rate:          0.920
  • Style false negatives:   16
  • Detoxify hit rate:       1.000
  • Detoxify false negatives:0

♦ COMBINED DETECTOR PERFORMANCE
  • Combined hit rate:       1.000
  • Combined false negatives:0

♦ CONFUSION MATRICES (Few-shot model)
  Clean exemplars:
[[90  8]
 [ 3 99]]

  Poisoned exemplars:
[[ 93   5]
 [  2 100]]

  After-defense exemplars:
[[ 93   5]
 [  2 100]]
```

```
========================================
DETECTOR EVALUATION SUMMARY — Jigsaw
========================================

♦ FEW-SHOT CLASSIFIER PERFORMANCE
  • Clean accuracy:          0.880
  • Poisoned accuracy:       0.910
  • After-defense accuracy:  0.915
  • Flip rate:               0.040
  • Recovery rate:           0.955

♦ DETECTOR PERFORMANCE (Individual)
  • Lexical hit rate:        1.000
  • Lexical false negatives: 0
  • Style hit rate:          0.710
  • Style false negatives:   58
  • Detoxify hit rate:       1.000
  • Detoxify false negatives:0

♦ COMBINED DETECTOR PERFORMANCE
  • Combined hit rate:       1.000
  • Combined false negatives:0

♦ CONFUSION MATRICES (Few-shot model)
  Clean exemplars:
[[159  22]
 [  2  17]]

  Poisoned exemplars:
[[165  16]
 [  2  17]]

  After-defense exemplars:
[[166  15]
 [  2  17]]
```

```
========================================
DETECTOR EVALUATION SUMMARY — IMDb
========================================

♦ FEW-SHOT CLASSIFIER PERFORMANCE
  • Clean accuracy:          0.970
  • Poisoned accuracy:       0.965
  • After-defense accuracy:  0.970
  • Flip rate:               0.005
  • Recovery rate:           1.000

♦ DETECTOR PERFORMANCE (Individual)
  • Lexical hit rate:        1.000
  • Lexical false negatives: 0
  • Style hit rate:          0.045
  • Style false negatives:   191
  • Detoxify hit rate:       0.995
  • Detoxify false negatives:1

♦ COMBINED DETECTOR PERFORMANCE
  • Combined hit rate:       1.000
  • Combined false negatives:0

♦ CONFUSION MATRICES (Few-shot model)
  Clean exemplars:
[[95  3]
 [ 3 99]]

  Poisoned exemplars:
[[95  3]
 [ 4 98]]

  After-defense exemplars:
[[95  3]
 [ 3 99]]
```

CONCLUSION

The data impacted and attack style utilized for running this study indicates how the data will affect the influence of backdoor triggers on few-shot classification tasks. Each attack type produced different results for each dataset used in the experiment using the same poisoning strategy. Jigsaw's representative quality became progressively worse with increasing percentages of poisoned exemplars on the dataset. Alternatively, IMDb's poisoned examples supported improved performance. Finally, SST-2 remained stable across both conditions. From these results, we can infer that the behavior of backdoor-like trigger mechanisms differs depending on the specific task and that in some cases the use of poisoned exemplars or examples may incidentally optimize a model's prompts. The difficulty to generalize adversarial behavior across different datasets in few-shot situations is supported by the different effects generated from using poisoned exemplars, thus it is essential to evaluate any type of attack used across all datasets rather than merely presupposing identical effects across all datasets.

Our work also shows that lightweight defenses can provide meaningful protection with minimal overhead. For the specific tasks that were adversely impacted by poisoning, the use of certain approaches, such as shuffling exemplars, removing invisible characters, and creating a uniform text format, stabilized the behavior of the classifiers and re-established their accuracy. Even though we could not conduct complete style evasion attacks during the duration of the project, the information derived from our efforts raises many significant questions that warrant further investigation, including: (1) why do some datasets experience positive effects from poisoning? (2) How does the architecture of a model contribute to its robustness? and (3) will similar defenses protect against more advanced attack methods, such as paraphrasing or stylistic rewriting? The pursuit of answers to these questions will be important in advancing our understanding of corruption and backdoors in security-focused few-shot classification pipelines.

WHAT WE LEARNED

Highly orchestrated adversarial attacks pose an imminent and significant threat to contemporary cybersecurity systems. This project provided a snippet of how modern classifiers behave when faced with different types of adversarial attacks. The model architecture dictates its performance and significance; A modern, instruction-tuned model, will obviously be more resilient to digital data corruption than an outdated model that lacks information and thorough training. Implementing detectors and lightweight defenses helps minimize model degradation and introduce layers of protection to the security infrastructure.

REFERENCES

Wallace, E., Feng, S., Kandpal, N., Gardner, M., & Singh, S. (2019). *Universal adversarial triggers for attacking and analyzing NLP*. In Proceedings of EMNLP. https://aclanthology.org/D19-1221.pdf (Accessed Oct 08, 2025).

Zhao, S., Wen, J., Tuan, L. A., Zhao, J., & Fu, J. (2023). *Prompt as Triggers for Backdoor Attack: Examining the Vulnerability in Language Models*. arXiv preprint. https://arxiv.org/abs/2305.01219 (Accessed Oct 08, 2025).

Zhang, X., et al. (2020). *Trojaning language models for fun and profit*. arXiv preprint. https://arxiv.org/abs/2008.00312 (Accessed Oct 08, 2025).

Krishna, K., Song, Y., Karpinska, M., Wieting, J., & Iyyer, M. (2023). *Paraphrasing evades detectors of AI-generated text, but retrieval is an effective defense*. NeurIPS. https://arxiv.org/abs/2303.13408 (Accessed Oct 08, 2025).

Zhang, T. (2024). *Few-shot Backdoor Removal via Adversarial Prompt Tuning (PromptFix)*. NAACL. https://aclanthology.org/2024.naacl-long.177.pdf (Accessed Oct 08, 2025).

nyu-mll. (n.d.). *SST-2 (Stanford Sentiment Treebank, GLUE subset)*. Hugging Face. https://huggingface.co/datasets/nyu-mll/glue/viewer/sst2 (Accessed Oct 08, 2025).

Google. (n.d.). *Jigsaw Toxic Comment Classification Challenge*. Kaggle. https://www.kaggle.com/competitions/jigsaw-toxic-comment-classification-challenge (Accessed Oct 08, 2025).

Google. (n.d.). *Jigsaw Toxicity Prediction Dataset*. Hugging Face. https://huggingface.co/datasets/google/jigsaw_toxicity_pred (Accessed Oct 08, 2025).

Lakshmi, 25npathi. (n.d.). *IMDb Dataset of 50K Movie Reviews*. Kaggle. https://www.kaggle.com/datasets/lakshmi25npathi/imdb-dataset-of-50k-movie-reviews (Accessed Oct 08, 2025).

Ganitkevitch, J., Van Durme, B., & Callison-Burch, C. (2013). *PPDB: The Paraphrase Database*. In Proceedings of NAACL (pp. 758-764). https://aclanthology.org/N13-1092/ (Accessed Oct 08, 2025). Dataset mirror available via Ellie Pavlick's site: https://cs.brown.edu/people/epavlick/data.html (Accessed Oct 08, 2025).

Wieting, J., Bansal, M., Gimpel, K., & Livescu, K. (2019). *PARANMT-50M: Pairs of Paraphrases from Neural Machine Translation*. ACL. https://github.com/facebookresearch/ParaNMT-50M (Accessed Oct 08, 2025).

OpenAI. (n.d.). *OpenAI API Reference*. OpenAI. https://platform.openai.com/docs/api-reference (Accessed Oct 08, 2025).

Wallace, E., Feng, S., Kandpal, N., Gardner, M., & Singh, S. (2019). *Universal Adversarial Triggers for Attacking and Analyzing NLP*. EMNLP-IJCNLP. https://github.com/Eric-Wallace/universal-triggers (Accessed Oct 08, 2025).

Zhao, P., Yu, L., & Xu, H. (2023). *ProAttack: Prompt Attack Framework for Evaluating Large Language Models*. arXiv. https://arxiv.org/abs/2305.01219 (Accessed Oct 08, 2025).

Krishna, K., Song, Y., Karpinska, M., Wieting, J., & Iyyer, M. (2023). *Paraphrasing Evades Detectors of AI-Generated Text, but Retrieval Is an Effective Defense*. NeurIPS. https://github.com/martiansideofthemoon/ai-detection-paraphrases (Accessed Oct 08, 2025).

Unitary AI. (n.d.). *Detoxify: Toxic Comment Classification Model*. GitHub. https://github.com/unitaryai/detoxify (Accessed Oct 08, 2025).

Wu, J., Zhan, R., Wong, D. F., Yang, S., Yang, X., Yuan, Y., & Chao, L. S. (2024). *DetectRL: Benchmarking LLM-Generated Text Detection in Real-World Scenarios*. GitHub. https://github.com/NLP2CT/DetectRL (Accessed Oct 08, 2025).